

Unidad 4



PHP (III). sesiones y bases de datos

Apuntes realizados para la asignatura de FP Grado Superior:
Implantación de Aplicaciones Web
del ciclo Administración de Sistemas Informáticos en Red

Autor: Jorge Sánchez Asenjo (www.jorgesanchez.net)
Versión del documento: 2.01, Año 2013



Esta obra está bajo una licencia de Reconocimiento-NoComercial-CompartirIgual de Creative Commons.
Para ver una copia de esta licencia, visite: <http://creativecommons.org/licenses/by-nc-sa/3.0/deed.es>



Reconocimiento-NoComercial-CompartirIgual 2.5 España

Usted es libre de:



copiar, distribuir y comunicar públicamente la obra



hacer obras derivadas

Bajo las condiciones siguientes:



Reconocimiento. Debe reconocer los créditos de la obra de la manera especificada por el autor o el licenciador (pero no de una manera que sugiera que tiene su apoyo o apoyan el uso que hace de su obra).



No comercial. No puede utilizar esta obra para fines comerciales.



Compartir bajo la misma licencia. Si altera o transforma esta obra, o genera una obra derivada, sólo puede distribuir la obra generada bajo una licencia idéntica a ésta.

- Al reutilizar o distribuir la obra, tiene que dejar bien claro los términos de la licencia de esta obra.
- Alguna de estas condiciones puede no aplicarse si se obtiene el permiso del titular de los derechos de autor
- Apart from the remix rights granted under this license, nothing in this license impairs or restricts the author's moral rights.

Advertencia

Los derechos derivados de usos legítimos u otras limitaciones reconocidas por ley no se ven afectados por lo anterior.

Esto es un resumen legible por humanos del texto legal (la licencia completa) disponible en los idiomas siguientes:

Catalán Castellano Euskera Gallego

Para ver una copia completa de la licencia, acudir a la dirección
<http://creativecommons.org/licenses/by-nc-sa/2.5/es/legalcode.es>

índice

(4.1) control de sesiones en PHP	5
(4.1.1) limitaciones del protocolo http	5
(4.1.2) sesiones	5
(4.1.3) técnicas para almacenar el estado de una sesión	6
(4.1.4) uso de cookies	6
(4.1.5) uso de sesiones en PHP	9
(4.2) bases de datos	12
(4.2.1) ventajas de las bases de datos	12
(4.2.2) instalación y configuración de MySQL	13
(4.2.3) conexión a MySQL desde PHP	13
(4.2.4) cerrar la conexión con la base de datos	14
(4.2.5) captura de errores	14
(4.2.6) seleccionar base de datos	16
(4.2.7) tipos de datos MySQL	16
(4.2.8) ejecución de instrucciones SQL	17

(4)

PHP (III)

sesiones.

acceso a bases de datos

(4.1) control de sesiones en PHP

(4.1.1) limitaciones del protocolo http

El protocolo que se utiliza para navegar por la web es http. Se dice que http es un protocolo sin estado (o sin memoria). La razón es que no almacena información alguna sobre la petición o la respuesta, lo que provoca que cada petición sea independiente de la anterior, no se tiene en cuenta lo realizado en la petición anterior.

Al principio, los desarrollos web no estaban preocupados por esta circunstancia ya que, esencialmente, servían páginas estáticas que contenía información simple. Sin embargo en la actualidad necesitamos otorgar *memoria* a nuestros desarrollos para adaptar nuestras páginas web al usuario concreto o a las acciones que este realiza. Eso permite una experiencia más rica al usuario o usuaria.

(4.1.2) sesiones

Una sesión es la navegación que hace un determinado usuario o usuario por un sitio web. La sesión se refiere a todas las peticiones que realiza el usuario referidas al mismo sitio web. Una sesión se inicia cuando el usuario acude a una página del sitio web y finaliza cuando el usuario abandona el sitio.

Durante toda la sesión el usuario puede haber establecido preferencias y elegido opciones que nos interese recordar. Eso es lo que se denomina el *estado de una sesión*. Deberemos conseguir que esa información no se pierda para utilizarla tanto en beneficio del usuario como en el nuestro propio como propietarios del sitio.

(4.1.3) técnicas para almacenar el estado de una sesión

- **Uso de la dirección IP.** El array `$_SERVER` proporciona este dato del usuario entre otros (se puede usar `$_SERVER["REMOTE_HOST"]` o `$_SERVER["REMOTE_ADDR"]`). Eso en principio permite identificarle; pero no es un método seguro ya que el usuario/a puede acceder desde servidores proxy o utilizando otras tecnologías que no nos permitan identificar con seguridad su dirección IP.
- **Uso de variables ocultas de formulario.** Cada vez que el usuario realice acciones que supongan datos a almacenar (los productos que va comprando, el usuario y contraseña con el que se identifica,...) los podemos pasar mediante campos de formulario oculto o bien añadiéndoles a la cadena URL de cada enlace por el sitio. Es posible usarlo siempre que tengamos la capacidad de identificar de forma única la sesión y de ser pasar los datos de enlace en enlace. El problema es que el paso por GET (el más cómodo de mantener mediante esta técnica) muestra los parámetros y eso es tremendamente inseguro.
Además, por mucha pericia que tengamos con esta técnica, es muy pesada de utilizar y lastra el desarrollo del conjunto del sitio web.
- **Almacenar datos de sesión en bases de datos o ficheros en el servidor.** Permite incluso que el usuario abandone el sitio web y puede, sin embargo, mantener la sesión que abandonó. Lo malo es que si tenemos una enorme cantidad de usuarios en nuestro sitio web, se convierte en un imposible mantener el sitio con esta técnica.
- **Uso de cookies.** La idea es la misma que en la anterior, sólo que la información no se guarda en el servidor sino en un archivo en el cliente. Ese archivo es lo que se conoce como una cookie. El usuario no será ni siquiera consciente el almacenamiento de la cookie, pero en ella almacenaremos los datos de la sesión y podremos mantener esa información sobre el usuario.
Lo malo es que el usuario o el usuario pueden bloquear las cookies o borrarlas y eso está fuera de nuestro control por lo que es una técnica que tiene también sus riesgos.

(4.1.4) uso de cookies

Son indudablemente la opción más habitual para almacenar la información del usuario. Lo malo es lo ya comentado, el cliente de un sitio web puede prohibir o borrar las cookies almacenadas.

Las cookies son un archivo de texto que almacenan información sobre el usuario y que se guardan en el propio ordenador del usuario. Contiene esencialmente nombres de variables y valores (de tipo string) de las mismas. Esas variables son las que se usan para guardar la información sobre el usuario.

Las cookies se pasan en la cabecera de la petición http por lo que su información viaja en el mismo paquete. En el momento que el servidor decide grabar los datos de la cookie, esta viaja con el paquete en todas las peticiones y respuestas de esa sesión.

Los datos de la cookie viajan en la cabecera, por lo que el servidor debe almacenar dichos datos antes de hacer uso de cualquier instrucción en pantalla.

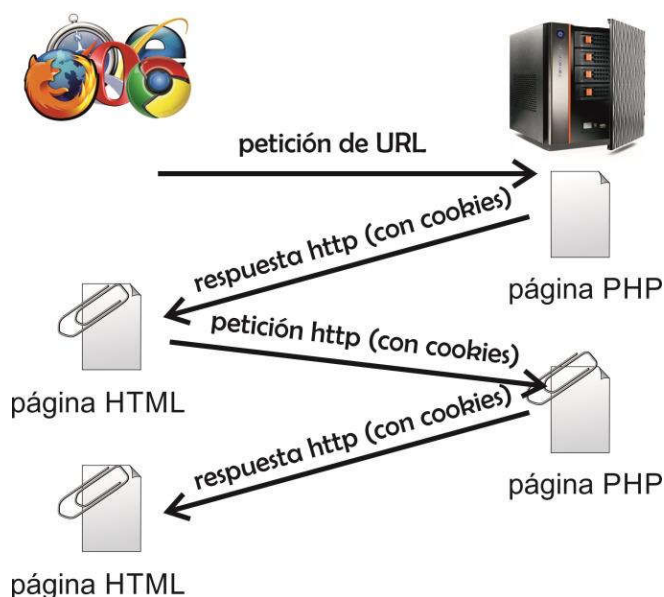


Ilustración 1, Funcionamiento de las cookies

setcookie

La función **setcookie** es la encargada de añadir (o borrar) una nueva cookie. Sintaxis:

```
setcookie(nombre [,valor [,expiración [,ruta [,dominio [, segura [sóloHttp ]]]]] )
```

Los parámetros son:

- **nombre**. Es el nombre que le damos a la cookie. Después podremos consultarle para poder saber su valor.
- **valor**. Valor que le damos a la cookie. Si no indicamos valor alguno, entonces estaremos borrando la cookie.
- **expiración**. Por defecto toma el valor cero, que significa que la cookie caduca en cuanto se cierre el navegador con el que se creó. Otra posibilidad es indicar una fecha en formato Unix. Por ejemplo **time()+600** indicaría que la cookie expira dentro de 10 minutos
- **dominio**. Si no se indica nada toma como dominio en el que se aplica la cookie el dominio actual, pero se puede indicar otro.
- **segura**. Por defecto vale **false**, si indicamos true; entonces indica que la cookie sólo se almacena si estamos comunicándonos mediante un protocolo seguro.
- **soloHttp**. Disponible desde la versión 5.2 de PHP. Indica que la cookie sólo está disponible mediante el protocolo **http** y no, por ejemplo, desde **JavaScript**. Se supone que evita inseguridades al utilizar cookies; pero es un tanto discutible.

Ejemplos:

```
setcookie("visitas",1);  
//graba la cookie visitas con valor 1 y que caduca con esta sesión  
setcookie("usuario","andrei",time()+60*60*24)  
//graba la cookie con nombre usuario que caducará al día siguiente
```

acceder a los datos de las cookies

El array global `$_COOKIE` contiene las cookies almacenadas en el equipo del cliente, de modo que para acceder al valor de una cookie se indicará el nombre de la cookie como índice de este array. Por ejemplo `$_COOKIE["visitas"]` devolvería el valor de la cookie con nombre `visitas` (suponiendo que esté definida dicha cookie). Ejemplo de lectura de una cookie:

```
if(isset($_COOKIE["visitas"])){  
    echo "esta es tu visita número".$_COOKIE["visitas"];  
}
```

borrar cookies

Cuando se vuelve a utilizar la función `setcookie` indicando el nombre de una cookie ya existente y un nuevo valor, el nuevo valor sobrescribe el anterior valor que tuviera dicha cookie.

Bajo la misma idea si usamos `setcookie` indicando un nombre de cookie existente y el valor cero o `false`, entonces se eliminará la cookie con ese nombre. Ejemplo:

```
setcookie("visitas",false); //elimina la cookie de nombre visitas
```

Podemos también eliminar cookies más antiguas indica una fecha de caducidad anterior a la actual, por ejemplo:

```
setcookie("visitas",false,time()-60*60*24);  
//elimina la cookie de nombre visitas con fecha de caducidad de un día  
//hasta este momento
```

almacenar datos binarios

En la cookies no se pueden almacenar datos binarios, sólo strings. Es decir no es válido el código:

```
setcookie("notas",array(9,7,6,5)); //inválido: datos binarios
```

Por ello todos los programadores para almacenar datos binarios (como los arrays), crean mecanismos para convertir lo binario en texto. En general al proceso de convertir los datos en un formato almacenable se le conoce como **serializar** (traducción excesivamente literal del inglés `serialize`).

De hecho PHP incorpora una función que realiza automáticamente este proceso, que se llama precisamente `serialize`. A esta función se le pasa un dato binario y le convierte en texto. De ese modo el array puede ser almacenado mediante la instrucción:

```
setcookie("notas",serialize(array(9,7,6,5))); //inválido: datos binarios
```

El problema ahora es que cuando lo queramos leer, tenemos el problema contrario: es decir convertir el texto en el binario correspondiente. Ese proceso es incluso más difícil; pero PHP lo facilita mediante la función contraria: `unserialize`. Para extraer el array del ejemplo usaríamos:

```
$variableArray=unserialize($_COOKIE["notas"]);
```


(4.1.5) uso de sesiones en PHP

ventajas y desventajas

Comparadas con las cookies ofrecen estas ventajas

- Pueden funcionar aunque el usuario/a desactive la posibilidad de cookies (podrían pasar el identificador de sesión mediante GET).
- Almacenan más información que una cookie y además son capaces de almacenar datos binarios.
- Son más seguras puesto que los datos que se almacenan de la sesión lo hacen en el servidor y no en el cliente. En principio la seguridad de los servidores es mayor que la de los ordenadores de los usuarios

Desventajas:

- Son susceptibles al ataque conocido como **secuestro de sesión** (*session hijacking*), mediante el cual un usuario se puede hacer con el identificador de sesión de otro usuario y hacerse pasar por él.
- El identificador de sesión se almacena mediante cookies, por lo que si no nos damos cuenta podríamos tener la misma dependencia con la configuración que tenga el usuario en el navegador.

funcionamiento

El manejo de sesiones parte de asignar un identificador a cada sesión. PHP dispone de la posibilidad de calcular dicho número de sesión para que sea único en cada sesión de usuario. Tras lo cual podemos identificar de formas unívoca a cada sesión. Ese identificador es el que se relacionará con los datos del usuario.

Los pasos son los siguientes:

- (1) Se genera un id aleatorio. Para ello se invoca a la función **session_start()**.
- (2) Se almacena cada dato de la sesión. Se asigna un nombre y un valor a cada dato de la sesión que deseemos almacenar. Para ello disponemos del array **\$_SESSION**, cuyos índices son los nombres de las variables de sesión a las cuales asignamos valores. Los índices son strings, pero (a diferencia de lo que ocurre con las cookies), los valores pueden ser binarios (incluso podemos almacenar arrays directamente)
- (3) En una página en la que deseemos recoger los datos almacenados de las sesiones, bastará con llamar a **session_start()** para poder recoger los valores de la sesión.
- (4) Mediante el array **\$_SESSION** podremos leer o escribir nuevos datos en la sesión

En realidad las sesiones son más sencillas de manejar que las cookies.

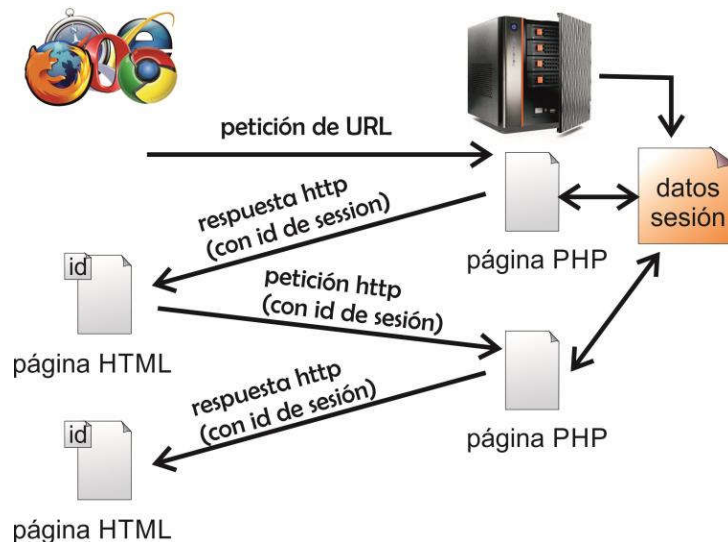


Ilustración 2, Funcionamiento de las sesiones PHP

La cuestión es ¿Cómo se pasa el identificador de sesión de petición en petición? Disponemos de dos posibilidades:

- **Mediante una cookie.** Se crea una cookie cuyo nombre es el nombre de la sesión y cuyo valor es el identificador de la sesión. En PHP es la opción por defecto. Lo malo es que el usuario puede prohibir utilizar cookies; lo bueno es que es más difícil obtener el identificador de sesión.

Para que esta opción sea la que funcione (si no fuera así) hay que modificar el archivo de configuración php.ini y colocar estos valores:

- `session.use_cookies 1`
- `session.use_trans_id 0`

- **Mediante GET.** En este caso se pasa el identificador de la sesión acompañando a cada petición mediante el uso de una cadena de consulta GET. Es decir que si nuestra sesión tiene el nombre de **PHPID** y el identificador fuera el **8FR45237A89** y estamos con la sesión ya iniciada y en la URL <http://prueba.com/pog1.php> la URL que tendremos será:

<http://prueba.com/pog1.php?PHPID=8FR45237A89>

Para que sea esta la opción que está funcionando, deberemos modificar el archivo php.ini y hacer estos cambios:

- `session.use_cookies 0`
- `session.use_trans_id 1`

Normalmente la sesión caduque en cuanto el usuario abandona el navegador. Se puede hacer (sobre todo si el identificador es una cookie) que dure más, pero no es lo recomendable. Las sesiones deben caducar cuando el usuario cierra la sesión, de otro modo hay más posibilidad de inseguridad y además es la forma habitual de usar sesiones. Si necesitamos esa perdurabilidad de la sesión, lo lógico es usar cookies o bases de datos.

iniciar sesión

Una sesión comienza en la primera página PHP que invoque a la función **session_start**. Esta función no tiene argumentos, simplemente inicia la sesión (si no ha sido iniciada ya) lo cual implica:

- (1) Crear un identificador aleatorio de sesión
- (2) Crear el archivo en el que se almacenarán los datos de la sesión en el servidor
- (3) Crear la variable global **\$_SESSION**, array en el que se almacenarán los datos de la sesión.

Si la sesión ya había sido iniciada cuando se invocó a **sesión_start**, entonces no se crea una nueva sesión sino que:

- (1) Se sigue utilizando la sesión anterior
- (2) Permite el uso del array **\$_SESSION** que contendrá los datos de la sesión

Con lo cual todas las páginas que quieran utilizar la sesión iniciada tienen que invocar a **session_start**.

obtener datos de la sesión

Dos funciones permiten obtener (y cambiar) los datos de la sesión

■ **session_id**. Si la invocamos sin parámetros, devuelve el identificador de sesión actual. Admite que le pasemos un identificador de sesión nosotros, con lo que ese se convierte en el identificador de la sesión actual. Poner nosotros números de sesión propios es peligroso en cuanto a que si no tenemos un buen algoritmo de cálculo del identificador podríamos tener dos sesiones

■ **session_name**. Si la invocamos sin parámetros, devuelve el nombre de sesión actual (por defecto PHPID). Admite que le pasemos un nombre de sesión nosotros. Esto es útil para que no sea tan claro que estamos usando sesiones PHP ya que el nombre PHPID es conocido por todos los programadores PHP.

La directiva **session.name** en el archivo de configuración **php.ini** permite asignar otro nombre por defecto para las sesiones.

usar variables de sesión

Para almacenar un dato de sesión basta usar esta sintaxis:

```
$_SESSION["clave"]=valor
```

Para recoger un valor almacenado se utiliza la misma clave en el array. Como hemos comentado es posible incluso almacenar datos binarios con lo que las sesiones son bastante más versátiles que las cookies.

borrar datos de sesión

Si deseamos eliminar un dato de la sesión basta con usar la función PHP de eliminación de variables **unset**. De modo que si deseamos eliminar una variable de sesión llamada **visitas**, haremos:

```
unset($_SESSION["visitas"])
```

Para eliminar todos los datos, hay que hacer esta secuencia:

```
unset($_SESSION);  
$_SESSION=array();//imprescindible, sino la sesión queda inutilizable
```

eliminar la sesión entera

Eliminar la sesión en sí consiste en:

```
session_start();//si no la hubieramos llamado ya  
unset($_SESSION);//elimina el array, sino queda gastando memoria  
session_destroy();//función de eliminación en sí de la sesión
```

(4.2) bases de datos

(4.2.1) ventajas de las bases de datos

El uso de bases de datos es imprescindible en cualquier aplicación web de hoy en día. Algunas razones para utilizarlas son:

- **Proporcionan almacenamiento permanente.** Es decir, los datos que se almacenan resisten entre sesiones con lo cual son un método ideal cuando la información no se desea eliminar en ningún momento.
- **Añaden una capa más de seguridad.** Los datos están fuera del alcance directo del usuario ya que no se almacenan en las peticiones http. Los datos se encuentran en un servidor de base de datos con el que se comunica el servidor web. En esta comunicación no interviene el usuario. Aunque hay técnicas para intentar acceder a la base de datos usando trucos, lo cierto es que las bases de datos son un software más preparado para la seguridad que si nosotros gestionáramos directamente la información
- **Proporcionan herramientas avanzadas de gestión de datos y usuarios.** Los Sistemas Gestores de Bases de Datos son un software especializado en la gestión de datos, permite hacer consultas avanzadas, crear diferentes vistas y permitir su uso a ciertos usuarios, copias de seguridad, transacciones,...
- **Uso del lenguaje SQL.** El lenguaje más conocido para el manejo de datos en bases de datos relacionales, está disponible y esto significa que podremos usar toda su potencia en nuestras páginas web.

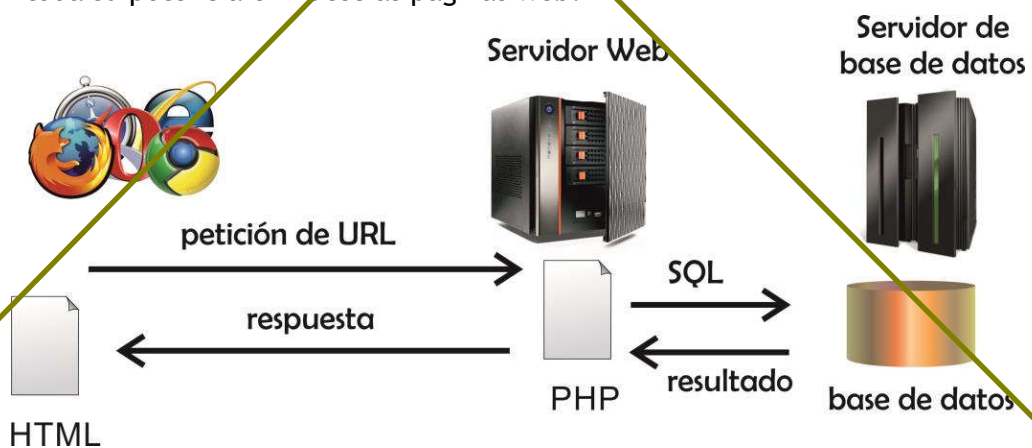


Ilustración 3, Funcionamiento de las bases de datos en PHP